

SansDIR: A Keyboard-Driven Terminal Application for Navigating, Inspecting, and Plotting Small-Angle Neutron Scattering Data

Changwoo Do

September 2026

Draft. Document has not been reviewed and approved for public release.

DOCUMENT AVAILABILITY

Online Access: US Department of Energy (DOE) reports produced after 1991 and a growing number of pre-1991 documents are available free via <https://www.osti.gov/>.

The public may also search the National Technical Information Service's [National Technical Reports Library \(NTRL\)](#) for reports not available in digital format.

DOE and DOE contractors should contact DOE's Office of Scientific and Technical Information (OSTI) for reports not currently available in digital format:

US Department of Energy
Office of Scientific and Technical Information
PO Box 62
Oak Ridge, TN 37831-0062

Telephone: (865) 576-8401

Fax: (865) 576-5728

Email: reports@osti.gov

Website: <https://www.osti.gov/>

This report was prepared as an account of work sponsored by an agency of the United States Government. Neither the United States Government nor any agency thereof, nor any of their employees, makes any warranty, express or implied, or assumes any legal liability or responsibility for the accuracy, completeness, or usefulness of any information, apparatus, product, or process disclosed, or represents that its use would not infringe privately owned rights. Reference herein to any specific commercial product, process, or service by trade name, trademark, manufacturer, or otherwise, does not necessarily constitute or imply its endorsement, recommendation, or favoring by the United States Government or any agency thereof. The views and opinions of authors expressed herein do not necessarily state or reflect those of the United States Government or any agency thereof.

Neutron Scattering Division

**SANS DIR: A KEYBOARD-DRIVEN TERMINAL APPLICATION FOR
NAVIGATING, INSPECTING, AND PLOTTING
SMALL-ANGLE NEUTRON SCATTERING DATA**

Changwoo Do

September 2026

Prepared by
OAK RIDGE NATIONAL LABORATORY
Oak Ridge, TN 37831
managed by
UT-BATTELLE LLC
for the
US DEPARTMENT OF ENERGY
under contract DE-AC05-00OR22725

CONTENTS

LIST OF FIGURES	v
LIST OF TABLES	vi
LIST OF ABBREVIATIONS	vii
FOREWORD	vii
ABSTRACT	1
1. INTRODUCTION	1
1.1 MOTIVATION	1
1.2 DESIGN PRINCIPLES	2
1.3 HERITAGE AND RELATED TOOLS	2
1.4 SCOPE OF THIS REPORT	3
2. INSTALLATION AND DEPLOYMENT	4
2.1 ZERO-INSTALL DEPLOYMENT ON THE ANALYSIS CLUSTER	4
2.2 PORTABILITY OF THE LAUNCHER	4
2.3 DEPENDENCIES	4
2.4 VERIFYING AN INSTALLATION	5
3. SOFTWARE ARCHITECTURE	6
3.1 LAYERED STRUCTURE	6
3.2 THE COMMAND REGISTRY	6
3.3 DISPATCH PATHS	7
3.4 KEY BINDINGS AS DATA	7
3.5 THE COMMAND LINE PARSER	8
3.6 STARTUP BUDGET AND LAZY IMPORTS	8
3.7 CONCURRENCY MODEL	9
3.8 REPOSITORY LAYOUT	9
4. THE USER INTERFACE	10
4.1 SCREEN ANATOMY	10
4.2 PANES AND ACTIVATION	10
4.3 FILE LISTING AND COLOR CODING	11
4.4 TAGGING	11
4.5 COMMAND LINE AND HISTORY	11
4.6 HELP OVERLAY	11
5. NAVIGATION AND FILE OPERATIONS	12
5.1 NAVIGATION	12
5.2 FILE OPERATION SEMANTICS	12
5.3 REFRESH BEHAVIOR	13
5.4 ARCHIVING AND EMAIL	13
5.5 SHELL ESCAPE	13
6. PLOTTING	14
6.1 PLOTTING THE SELECTION	14
6.2 ONE-DIMENSIONAL PLOTS	14
6.3 TWO-DIMENSIONAL PLOTS	14
6.4 DETECTOR HEAT MAPS FROM NEXUS	15
6.5 GENERIC TABULAR PLOTS	15
7. METADATA INSPECTION AND EXTRACTION	17
7.1 SINGLE-FILE TREE BROWSER	17
7.2 KEYWORD SEARCH	17

7.3	BATCH EXTRACTION	18
7.4	OUTPUT MODES	18
7.5	A WORKED EXAMPLE	18
8.	CATALOG INTEGRATION	20
8.1	AUTHENTICATION	20
8.2	SEARCH MODEL AND CACHING	20
8.3	RUN CATALOG PANE	20
9.	DETECTOR MASK CREATION	21
9.1	ARCHITECTURAL CONSTRAINTS	21
9.2	INTERACTIVE EDITOR	21
9.3	COMMAND-LINE MASK CREATION	22
9.4	OUTPUT FORMATS AND REDUCTION COMPATIBILITY	22
10.	NON-INTERACTIVE COMMAND-LINE INTERFACE	23
10.1	LAUNCHING THE INTERFACE	23
10.2	BATCH METADATA EXTRACTION	23
10.3	MASK CREATION	23
11.	CONFIGURATION	24
12.	USANS REDUCTION	25
12.1	INSTRUMENT MODE	25
12.2	THE SETUP TABLE	25
12.3	INVOKING THE ENGINE	26
12.4	ONE KEY	26
12.5	A WORKED EXAMPLE	26
12.6	DESMEARING	27
13.	SUPPORTED DATA FORMATS	29
13.1	ASCII REDUCED DATA	29
13.2	NEXUS FILES	29
14.	TESTING AND QUALITY ASSURANCE	30
14.1	TEST SUITE	30
14.2	STATIC ANALYSIS AND STYLE	30
15.	PERFORMANCE	31
16.	SUMMARY	32
17.	REFERENCES	34
	APPENDIX A. COMMAND REGISTRY REFERENCE	A-1
	APPENDIX B. DEFAULT KEY MAP	B-1
	APPENDIX C. WORKED EXAMPLE SESSION	C-1

LIST OF FIGURES

Figure 1.	Layered architecture of sansdir.	6
Figure 2.	The help overlay, generated from the registry.	8
Figure 3.	The dual-pane interface.	10
Figure 4.	One-dimensional reduced data.	15
Figure 5.	Detector heat map from a raw NeXus event file.	16
Figure 6.	Keyword search in the HDF5 tree browser.	17
Figure 7.	Batch metadata extraction, end to end.	19
Figure 8.	USANS reduction, end to end, on one key.	27

LIST OF TABLES

Table 2.	Relocatable and fixed components of a shared installation	5
Table 3.	Runtime dependencies	5
Table 4.	Callers of the command registry	7
Table 5.	Package structure	9
Table 6.	File-kind color coding in the panes	11
Table 7.	Navigation keys	12
Table 8.	Function-key file operations	12
Table 9.	What p draws for each kind of file	14
Table 10.	Interactive mask editor controls	21
Table 11.	Recognized ASCII column layouts	29
Table 12.	Performance budgets	31
Table A-1.	Registered commands. A trailing question mark marks an optional parameter	A-2
Table B-1.	Default key bindings	B-2

LIST OF ABBREVIATIONS

API	application programming interface
CI	continuous integration
CLI	command-line interface
CSV	comma-separated values
DAS	data acquisition system
DOE	US Department of Energy
DOI	digital object identifier
GUI	graphical user interface
HDF5	Hierarchical Data Format version 5
HFIR	High Flux Isotope Reactor
IPTS	Integrated Proposal Tracking System
JSON	JavaScript Object Notation
LLM	large language model
mc	Midnight Commander
NFS	Network File System
NIST	National Institute of Standards and Technology
OAuth2	Open Authorization version 2
ORNL	Oak Ridge National Laboratory
PNG	Portable Network Graphics
REST	representational state transfer
SANS	small-angle neutron scattering
SNS	Spallation Neutron Source
SSH	Secure Shell
TSV	tab-separated values
TTL	time to live
TUI	terminal user interface
USANS	ultra-small-angle neutron scattering
XML	Extensible Markup Language

FOREWORD

This report documents **sansdir**, a terminal application developed in the Neutron Scattering Division at Oak Ridge National Laboratory (ORNL) to support the day-to-day data-handling work of small-angle neutron scattering (SANS) instrument scientists and users. The software is in production use for Spallation Neutron Source (SNS) SANS workflows and is installed on the analysis cluster, launched as `/SNS/EQSANS/shared/bin/sansdir`.

The report is written for two audiences. Sections 1 through 11 are a user manual: they describe what the program does, how to install and run it, and how each feature behaves. Sections 13 through 16 are a developer and reviewer reference: they document the internal architecture, the testing strategy, and measured performance. Readers who only want to use the program can stop after Section 11; the appendices provide condensed reference tables of every command and key binding.

ABSTRACT

SANS users at the SNS spend a substantial fraction of their beam-time and post-experiment effort on tasks that are neither data reduction nor data analysis: locating runs across Integrated Proposal Tracking System (IPTS) directories, triaging reduced data by inspecting plots, reading instrument logs out of NeXus files, exporting selected metadata to tables, and packaging results for collaborators. These tasks are performed over Secure Shell (SSH) on the analysis cluster, where graphical user interface (GUI) tools that require X11 forwarding respond slowly.

This report describes **sansdir**, a dual-pane, keyboard-driven terminal user interface (TUI) that consolidates these tasks into a single application requiring no display server. The program follows the interaction model of the MDIR and Norton Commander file managers: two independent directory panes, function-key actions, and file operations that default to moving data from the active pane to the opposite one. It adds SANS-specific capability: automatic classification and plotting of reduced one- and two-dimensional data, detector heat maps computed directly from raw NeXus event files without a Mantid runtime, a searchable Hierarchical Data Format version 5 (HDF5) tree browser, batch metadata extraction to comma-separated values (CSV) and tab-separated values (TSV), an interactive detector-mask editor whose output is compatible with the `drt sans` reduction pipeline, and catalog search through the ORNL OnCat service.

The central architectural decision is that every user-facing action is a typed command in a single registry. Key bindings, the interactive command line, and the non-interactive command-line interface (CLI) are all thin callers of that registry, which makes the interaction surface uniform, self-documenting, and testable, and which positions a future natural-language layer as an additional caller rather than a rewrite. The implementation is approximately 18,400 lines of Python across 58 modules, covered by 799 automated tests, and starts in well under the 300 ms budget.

1. INTRODUCTION

1.1 MOTIVATION

A small-angle neutron scattering (SANS) experiment at the Spallation Neutron Source (SNS) produces raw event-mode NeXus files, one per run, that are written into an Integrated Proposal Tracking System (IPTS) directory tree of the form `/SNS/EQSANS/IPTS-NNNNN/nexus/`. Reduction—performed by `drt sans` (Oak Ridge National Laboratory 2025) or by the automatic reduction service—emits one-dimensional $I(q)$ profiles, two-dimensional $I(q_x, q_y)$ arrays, transmission curves, and log files into a parallel `shared/` tree. A single proposal routinely accumulates hundreds of runs and several thousand derived files.

Between measurement and publication, the instrument scientist and the user perform a set of tasks that are repetitive, unglamorous, and poorly served by existing software:

- **Navigation.** Moving between IPTS directories, between the `nexus/` and `shared/` subtrees of one proposal, and between a proposal directory and a personal scratch area.
- **Triage.** Opening a reduced $I(q)$ file to check whether a measurement worked, comparing two or three curves on the same axes, or looking at a raw detector image to confirm beam-center and beam-stop placement.
- **Log inspection.** Reading sample environment values—temperature, shear rate, sample position—out of the data acquisition system (DAS) logs stored inside a NeXus file, either for one run or as a table across many runs.

- **Packaging.** Selecting a subset of results, compressing them, and emailing them to a collaborator.

None of these is difficult in isolation. What makes them costly is that they are performed dozens of times a day, from a terminal session on the analysis cluster, and that the available tools each address only one of them. Mantid Workbench (Arnold, Bilheux, Borreguero, et al. 2014) is a full analysis environment and is correspondingly heavy to launch for the purpose of glancing at one curve. A file manager handles navigation but knows nothing about SANS file formats. Reading DAS logs means writing a short `h5py` (Collette 2025) script. Any graphical user interface (GUI) tool launched over Secure Shell (SSH) with X11 forwarding responds with a latency that makes rapid iteration unpleasant.

The economic argument for **sansdir** is therefore not that it does something impossible, but that it collapses a dozen context switches per hour into a single keyboard-driven application that runs natively in the terminal the scientist is already using.

1.2 DESIGN PRINCIPLES

Four principles govern the implementation. They are recorded here because they are binding on future contributions, not merely descriptive.

1. **Speed first.** Every interaction must feel instantaneous. No blocking input/output is permitted on the user-interface thread; catalog queries, Hierarchical Data Format version 5 (HDF5) reads, and plot rendering run on worker threads or in separate processes.
2. **Keyboard-driven.** Every action has a single-key or short-chord binding. Mouse support is acceptable but never required.
3. **Predictable.** A user familiar with MDIR, Norton Commander, or Midnight Commander (mc)-style file managers (The Midnight Commander Development Team 2025) should be productive within minutes. This dictates two panes, function-key actions, and “operate from the active pane into the opposite pane” semantics.
4. **The command registry is the single source of truth.** Every user-facing action is a registered command with a name, typed parameters, a description, and examples. Key bindings, the command line, and the command-line interface (CLI) all dispatch through it. No key handler is ever wired directly to business logic.

1.3 HERITAGE AND RELATED TOOLS

The interaction model descends from MDIR, a DOS-era file manager, and from the Norton Commander family that includes mc. The defining feature of that family is the dual-pane layout: two independent directory views, exactly one of which is active, with copy and move operations defaulting to “from the active pane to the other pane.” This convention eliminates the source-and-destination dialog that dominates single-pane and graphical file managers, and it is the reason experienced Norton Commander users move files faster than mouse-driven equivalents. Preserving it is treated as a hard constraint rather than an aesthetic preference.

sansdir is complementary to, not a replacement for, the existing SNS software stack. It performs no data reduction; `drt sans` remains the reduction engine. It performs no model fitting. It does not replace Mantid Workbench for interactive analysis. Its scope is deliberately the layer *around* reduction and analysis: finding files, looking at them, extracting metadata from them, and moving them. Where the boundary is crossed—the detector-mask editor of Section 9 produces an input to reduction—the output format is defined by what `drt sans` consumes.

An earlier tool by the same author, `eqsanscli` (Do 2025), provided command-line access to OnCat catalog search. Its application programming interface (API) endpoints and authentication flow informed the OnCat client described in Section 8; that client was written fresh for this program rather than by copying `eqsanscli`'s source code into it.

1.4 SCOPE OF THIS REPORT

This report describes version 0.10.2. Section 2 covers installation and deployment. Section 3 describes the software architecture, with emphasis on the command registry. Sections 4 through 9 document the user-facing feature set. Section 10 covers non-interactive use, and Section 11 the configuration file. Sections 13 through 15 cover the supported data formats, the test suite, and measured performance. Section 16 summarizes the work and states the limitations that bound it.

2. INSTALLATION AND DEPLOYMENT

2.1 ZERO-INSTALL DEPLOYMENT ON THE ANALYSIS CLUSTER

The primary deployment target is the Oak Ridge National Laboratory (ORNL) analysis cluster, where a released, non-editable install lives at `/SNS/EQSANS/shared/script/sansdir-stable` together with a self-contained Python virtual environment, and is exposed through the shared tools directory as `/SNS/EQSANS/shared/bin/sansdir`. Every cluster user has read access to those paths, so no per-user installation step is required. The program is launched by running that command from any terminal on the cluster—a login node, a web-terminal session, or an SSH connection:

```
/SNS/EQSANS/shared/bin/sansdir  
/SNS/EQSANS/shared/bin/sansdir /SNS/EQSANS/IPTS-12345/shared
```

The first form opens the interface in the current working directory. The optional second argument is a starting path: the directory the left pane opens into, so a user can jump straight to a proposal’s `shared` folder without navigating there by hand. The right pane mirrors it initially. Any readable directory is accepted; a path under `/SNS/USANS` additionally starts the program in ultra-small-angle neutron scattering (USANS) mode (Section 12).

To avoid typing the full path, users are advised to create a symbolic link in a directory already on their `PATH`. On the analysis nodes both `~/bin` and `~/local/bin` are on the default `PATH`:

```
mkdir -p ~/bin  
ln -s /SNS/EQSANS/shared/bin/sansdir ~/bin/sansdir  
sansdir --version
```

Alternatively, the shared `bin` directory can be prepended to `PATH` in the user’s shell profile.

2.2 PORTABILITY OF THE LAUNCHER

The entry-point script may also simply be copied anywhere on the filesystem and executed from there. This works because the script’s shebang line names an *absolute* interpreter path inside the shared virtual environment, so the kernel executes the bundled Python regardless of where the script file itself resides. The resulting execution chain is:

```
[user’s copy of the script]  
-> /gpfs/.../sansdir/.venv/bin/python      (via absolute shebang)  
-> /gpfs/.../sansdir/src/sansdir/cli.py   (via the venv’s editable install)
```

Two consequences follow. First, the copy is independent of the user’s active conda environment, current working directory, and whichever Python happens to be first on `PATH`. Second, a *symbolic link is preferable to a copy* in practice: when the shared environment is rebuilt, every user holding a symlink picks up the new build automatically, whereas holders of copies do not.

Table 2 summarizes which components must remain on the shared mount and which are freely relocatable.

2.3 DEPENDENCIES

Python 3.10 or newer is required. Table 3 lists the runtime dependencies and the role each plays. The dependency set is deliberately small; proposals to add libraries—in particular `pandas`, `scipy`, or any character-cell plotting library—require justification against the startup budget of Section 3.6.

Table 2. Relocatable and fixed components of a shared installation

Path	Status	Note
.venv/	Fixed	Bundled Python interpreter and dependencies.
src/sansdir/	Fixed	Source tree that the editable install points to.
.venv/bin/sansdir	Portable	Python entry point; shebang is absolute.
bin/sansdir	Portable	Shell launcher; self-locates relative to the shared root.

Table 3. Runtime dependencies

Package	Minimum	Role
textual	0.80	Terminal user interface (TUI) framework; asynchronous, themeable (McGugan 2025).
click	8.1	CLI subcommand dispatch (Ronacher and The Pallets Team 2025).
matplotlib	3.7	All plotting (Hunter 2007).
numpy	1.24	Array input/output for ASCII and NeXus data (Harris, Millman, Walt, et al. 2020).
h5py	3.9	NeXus and HDF5 access (Collette 2025).
httpx	0.27	Asynchronous representational state transfer (REST) client for OnCat.
send2trash	1.8	Recoverable delete, with a fallback for cluster filesystems that provide no trash directory.
tomli	2.0	Tom’s Obvious Minimal Language parsing on Python 3.10 only; Python 3.11 and newer use the standard-library tomllib.

Note: Optional extras are `qt` (installs PyQt5 for an interactive matplotlib backend), `tk` (a marker extra; the `tkinter` module ships with most Python builds), `llm` (the Anthropic client for the planned natural-language layer), and `dev`.

2.4 VERIFYING AN INSTALLATION

```
sansdir --version      # prints the version and exits
sansdir --help         # subcommands and worked examples
sansdir extract --help # per-subcommand help
```

A successful `--version` invocation confirms that the interpreter, package, and entry point resolve correctly. It deliberately does not import Textual, matplotlib, or h5py, so it also serves as a check that the lazy-import discipline of Section 3.6 is intact.

3. SOFTWARE ARCHITECTURE

3.1 LAYERED STRUCTURE

The program is organized in four layers, shown in Figure 1. Entry points at the top translate user intent into a command name and arguments. The command registry in the middle is the sole dispatch mechanism. The service layer below it contains all business logic, grouped by concern. External resources sit at the bottom.

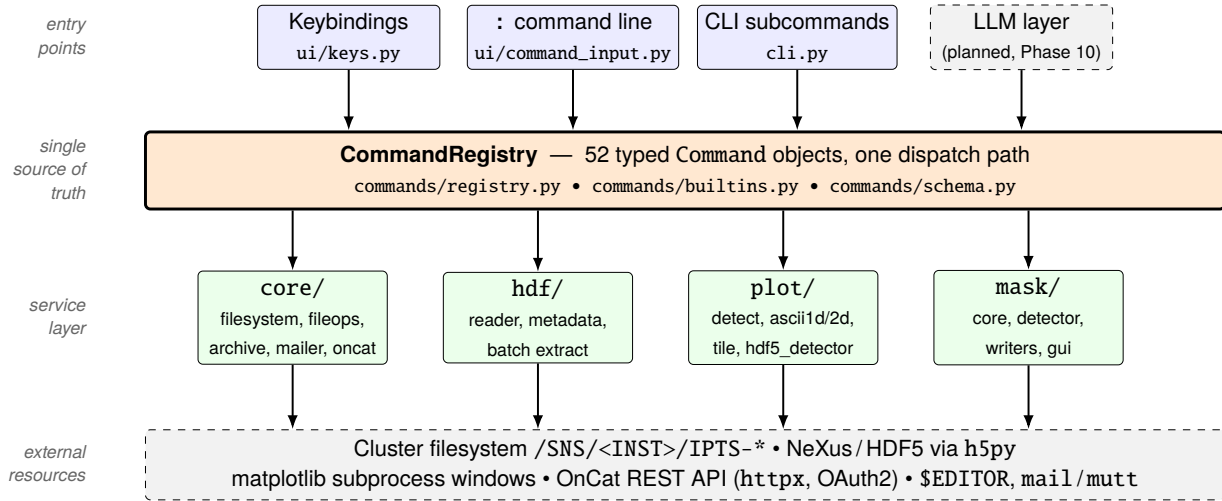


Figure 1. Layered architecture of sansdir. Every entry point—key bindings, the interactive command line, CLI subcommands, and the planned natural-language layer—converges on a single command registry. The application shell owns no business logic; the service layer below the registry contains all of it.

The Textual application object owns presentation state only: which pane is active, what the cursor and tags are, which modal screen is on the stack. It contains no file, plotting, or catalog logic. This separation is what makes the service layer testable without instantiating a terminal application, and it is enforced by code review.

3.2 THE COMMAND REGISTRY

The registry is the most consequential design decision in the project. Every user-facing action—change directory, tag, copy, plot, extract metadata, search the catalog, create a mask—is represented as a `Command` object holding a dotted name, a one-sentence description, a tuple of typed parameters, a handler, optional aliases and examples, and a `danger` flag marking destructive operations. Parameters are themselves typed objects with a name, a type drawn from a fixed vocabulary (`path`, `glob`, `string`, `int`, `float`, `bool`, `enum`, `files`), a description, a required flag, a default, and—for enumerations—a list of permitted values.

Validation is performed at construction time and at dispatch time. A parameter declaring an unknown type, or an enumeration without choices, raises immediately when the command is built, so a malformed command definition fails at import rather than when a user first presses the key. At dispatch, unknown keyword arguments and missing required arguments both raise `TypeError` before the handler runs.

Handlers may be synchronous or asynchronous. `dispatch` is itself asynchronous and awaits the handler’s result if it is awaitable, so callers have exactly one calling convention regardless of how a particular handler

is implemented. A synchronous convenience wrapper exists for call sites that are not already inside an event loop; it raises rather than silently misbehaving if invoked from within one.

Version 0.10.2 registers 57 commands. Appendix A lists them in full.

3.3 DISPATCH PATHS

Four callers use the registry, as summarized in Table 4. There is intentionally no fifth path: no code in the user-interface layer calls a service function directly.

Table 4. Callers of the command registry

Caller	Path
Key binding	<code>ui/keys.py</code> maps F6 to <code>ui.copy_tagged</code> , resolves the active pane’s selection and the inactive pane’s directory into keyword arguments, and dispatches.
Command line	<code>ui/command_input.py</code> parses <code>cd /SNS/EQSANS</code> into the command name <code>nav.cd</code> and the argument path, then dispatches.
CLI	<code>sansdir extract -k ...</code> builds arguments and calls the same extraction code path used by the interactive dialog.
Large language model (LLM) layer	Planned. The registry already exports a JavaScript Object Notation (JSON) tool schema; a natural-language front end would emit command names with typed arguments, display them for confirmation, and dispatch.

Three properties follow from this discipline, and they are the practical justification for it. First, the interactive and non-interactive interfaces cannot drift apart, because they are the same code. Second, the help overlay is generated from registry metadata rather than maintained by hand, so it cannot become stale—Figure 2 shows the generated output. Third, the future natural-language layer is an additional caller rather than a reimplementation.

3.4 KEY BINDINGS AS DATA

The key map is a plain list of immutable `KeyBinding` records, each naming a key, a command, a description, and an optional *argument resolver*—a small named function that converts the running application’s state into the keyword arguments the command expects. For example, the resolver bound to **m** extracts the path under the cursor in the active pane and returns it as the `path` argument to `hdf.show_keys`.

Python allows defining a one-line helper function without giving it a name (a *lambda*), but a nameless function identifies itself in error messages only as `<lambda>`. Every resolver here is therefore an ordinary function with a descriptive name, so an error traceback and the help overlay can both say which resolver was involved. The default map contains 48 bindings, of which 38 are shown in the help overlay; the remainder are duplicate encodings of the same key. Duplicates exist because terminals disagree on how to report certain keys: the equals sign arrives as `equals_sign` on most terminals and as `=` on others, so both are registered and the redundant entry is hidden from help.

A test asserts that every binding names a command that exists in a fully bound registry, so a typo in the key map fails in continuous integration (CI) rather than at runtime.

Users may override or extend the map through the `[keys]` section of the configuration file. Entries naming an unknown command are dropped at startup with a notification rather than aborting the launch.

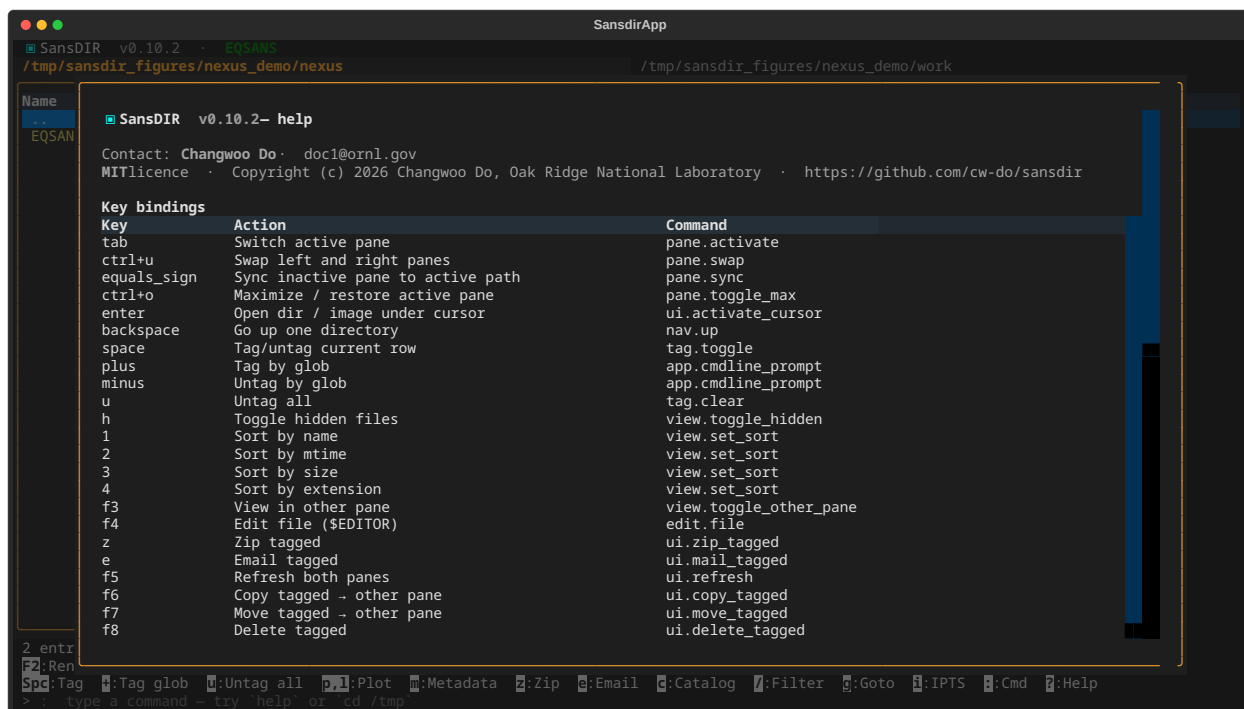


Figure 2. The help overlay, generated from the registry. Each row pairs a keystroke with its human-readable action and the underlying command name. The third column is what makes the mapping between the two interaction surfaces discoverable: any key can be re-issued as a typed command, and any command can be rebound to a key.

3.5 THE COMMAND LINE PARSER

The `:` line accepts registered command names and aliases with positional and keyword arguments. It performs no natural-language interpretation and contains no `eval` or `exec`; this is a deliberate safety boundary, and natural-language input is reserved for the future large language model (LLM) layer, which can emit only registry-defined commands with typed arguments.

Aliases keep the typed interface terse. `nav.cd` is reachable as `cd`, `hdf.batch_extract` as `extract`, `oncat.search` as `ipts`, and `shell.run` as `!`. Command history persists across sessions.

3.6 STARTUP BUDGET AND LAZY IMPORTS

The design budget is 300 ms from invocation to first paint. Re-measured after the interface grew the USANS reduction and desmearing features, the cold start for `sansdir -help` remains well inside that budget: the median end-to-end wall-clock time on an analysis node is about 120 ms, of which roughly 40 ms is Python interpreter start-up and about 40 ms is importing the command module; the best case is near 60 ms. The figure did not grow with the new features because the fast path imports none of them—a check confirms that `numpy`, `h5py`, `matplotlib`, `Textual`, and `httpx` are all absent from `sys.modules` after `import sansdir.cli`.

This is achieved by importing only `click` and the version constant at module scope in the CLI. `Textual`, `matplotlib`, `h5py`, `httpx`, and the registry itself are imported inside the subcommand bodies that need them. The practical effect is that `sansdir version` and `sansdir -help` never pay for the scientific stack, and that the terminal user interface (TUI)’s own startup imports `Textual` but still defers `matplotlib` and `h5py` until the first plot or metadata operation.

The budget is checked after any change that touches the import graph.

3.7 CONCURRENCY MODEL

Three mechanisms keep the interface responsive:

- **Thread offload.** Blocking library calls—principally `h5py` reads—run through `asyncio.to_thread`. The HDF5 key search of Section 7.2 is a representative case: walking a large raw NeXus file takes seconds, so the walk runs on a thread and the result is cached on the screen object for subsequent queries.
- **Textual workers.** Longer interactive operations run as background workers so that the event loop continues to service keystrokes. The mask editor’s launcher, for instance, awaits the editor subprocess in a worker and refreshes both panes when it exits with a save.
- **Subprocesses.** Plot windows run in a separate process, which is the only reliable way to give matplotlib its own GUI event loop without contending with Textual’s.

3.8 REPOSITORY LAYOUT

Table 5 lists the package structure. The implementation totals approximately 18,400 lines across 58 modules.

Table 5. Package structure

Path	Contents
<code>cli.py</code>	Click entry point; <code>tui</code> , <code>extract</code> , <code>mask</code> , <code>usans</code> , and <code>version</code> subcommands.
<code>app.py</code>	The Textual application shell.
<code>config.py</code>	Configuration loader for Tom’s Obvious Minimal Language (Preston-Werner et al. 2021) (“Tom’s Obvious Minimal Language,” the format’s real name), read with the standard-library <code>tomllib</code> .
<code>commands/</code>	<code>registry.py</code> (the Command model and dispatch), <code>builtins.py</code> (all 56 registrations), <code>parser.py</code> (the <code>:</code> line), <code>schema.py</code> (JavaScript Object Notation (JSON) tool schema export).
<code>core/</code>	<code>filesystem.py</code> (listing and sorting), <code>fileops.py</code> (copy, move, delete), <code>archive.py</code> , <code>mailer.py</code> , <code>oncat.py</code> , <code>history.py</code> , <code>instrument.py</code> (SANS/USANS mode resolution).
<code>hdf/</code>	<code>reader.py</code> (safe open, tree walk, value previews), <code>metadata.py</code> , <code>batch.py</code> (parallel extraction).
<code>plot/</code>	<code>detect.py</code> (file-kind classification), <code>ascii1d.py</code> , <code>ascii2d.py</code> , <code>tile.py</code> , <code>generic.py</code> , <code>image.py</code> , <code>hdf5_detector.py</code> , <code>backend.py</code> , <code>window.py</code> (the plot subprocess).
<code>usans/</code>	<code>grouping.py</code> (title blocks), <code>reconcile.py</code> (scan counts derived from disk), <code>table.py</code> (the setup comma-separated values (CSV)), <code>catalog.py</code> (table and notes), <code>runner.py</code> (invoking the external engine), <code>desmear.py</code> (truncated Abel inversion) and <code>desmear_io.py</code> (reading and writing curves). Pure; imports no interface or network module.
<code>mask/</code>	<code>core.py</code> (shapes and the mask builder), <code>detector.py</code> , <code>banktube.py</code> , <code>writers.py</code> , <code>mantid_writer.py</code> , <code>gui.py</code> , <code>api.py</code> .
<code>ui/</code>	<code>panel.py</code> , <code>pane_slot.py</code> , <code>keys.py</code> , <code>command_input.py</code> , <code>dialogs.py</code> , <code>hdf_tree.py</code> , <code>run_catalog.py</code> , <code>oncat_browser.py</code> , <code>help.py</code> , <code>statusbar.py</code> , <code>titlebar.py</code> , <code>pathbar.py</code> , <code>key_hint_bar.py</code> , <code>inline_viewer.py</code> , <code>prompt_panel.py</code> .

4. THE USER INTERFACE

4.1 SCREEN ANATOMY

Figure 3 shows a running session. From top to bottom the screen comprises a title bar carrying the program name and version; a two-up path bar showing each pane's working directory, with the active pane's path highlighted; the two file panes themselves; a status line reporting the entry count and the number of tagged files; a two-row key hint bar in the Norton Commander tradition; and the command line.

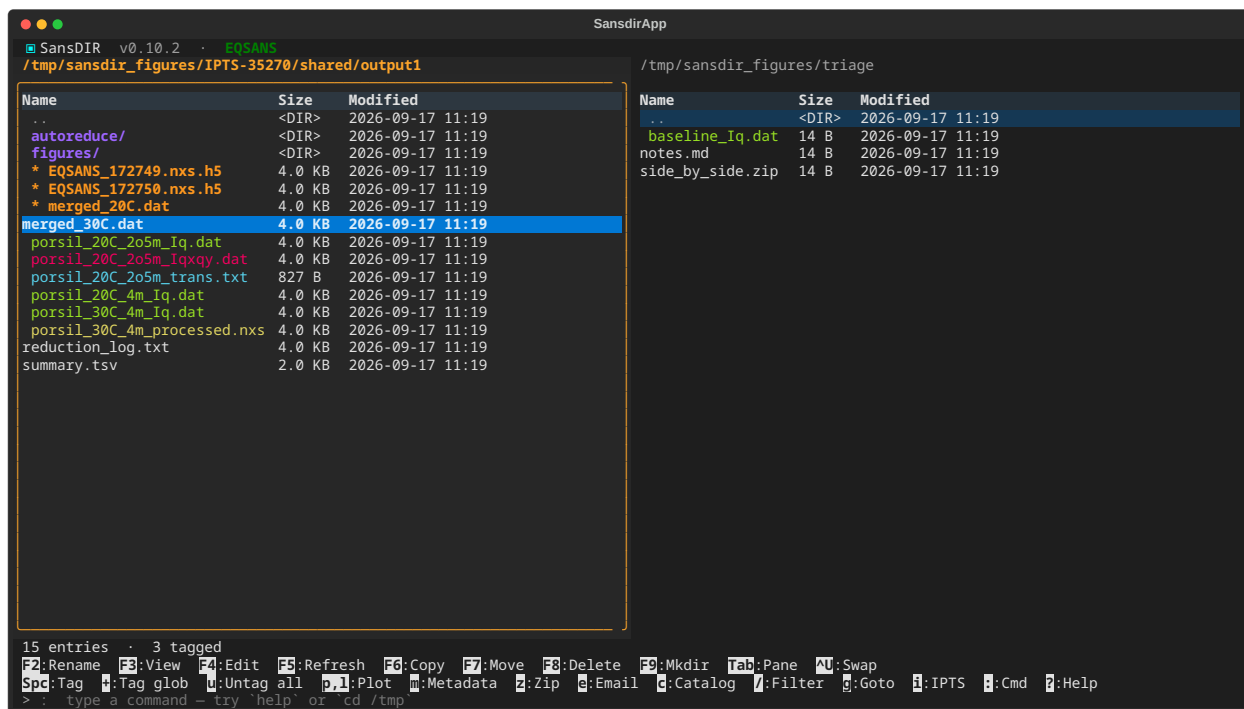


Figure 3. The dual-pane interface. The left pane is active, indicated by the highlighted border and path. Three files are tagged, shown by the leading asterisk and the bold rendering; the status line reports the count. File kinds are distinguished by color: directories, one-dimensional reduced data, two-dimensional reduced data, transmission files, and NeXus files each have a distinct hue.

The path bar rewrites the `/gpfs/neutronsfs/instruments` prefix to `/SNS`, which is how the same location is conventionally written and how users expect to see it, at no cost in accuracy since the two are the same mount.

4.2 PANES AND ACTIVATION

The two panes are instances of the same widget class holding independent state: working directory, cursor position, tag set, sort key, and filter. Exactly one is active at any moment, distinguished by border color. **Tab** switches activation.

Several commands manipulate the pair rather than a single pane. **Ctrl+U** swaps the two panes' contents, which is the fastest way to reverse the direction of a copy. **=** synchronizes the inactive pane to the active pane's directory, which is the usual preparation for a same-directory operation. **Ctrl+O** maximizes the

active pane to full width and restores it on a second press; this is a toggle, never a default, because the dual-pane layout is the point.

The run catalog described in Section 8 always occupies the right pane regardless of which pane was active when it was requested. This keeps the layout predictable: users learn that the catalog is on the right, and **Ctrl+U** swaps file panes without disturbing it.

4.3 FILE LISTING AND COLOR CODING

Each pane lists name, size, and modification time. Directories sort first. Sorting is by name, modification time, size, or extension, selected with the number keys **1** through **4**. Hidden files are suppressed by default and toggled with **h**. Symbolic links are followed but flagged.

Color encodes file kind so that a directory can be scanned at a glance (Table 6).

Table 6. File-kind color coding in the panes

Kind	Rendering
Directory	Bold blue
Symbolic link	Cyan
Iq.dat, one-dimensional reduced	Green
Iqxqy.dat, two-dimensional reduced	Magenta
trans.txt, transmission	Cyan
*.nxs.h5 and *.nxs, NeXus	Bright yellow
Executable	Bold red
Tagged row	Bold yellow with a leading asterisk

4.4 TAGGING

Tagging is the selection mechanism for every multi-file operation. **Space** toggles the tag on the row under the cursor. **+** prompts for a glob and tags every visible entry matching it; **-** untags by glob; **u** clears all tags in the active pane. Tags are per-pane and are cleared when that pane changes directory.

The universal rule is that an operation acts on the active pane’s tagged files if there are any, and otherwise on the single file under the cursor. This means the common single-file case requires no tagging at all, while the multi-file case requires no separate “select mode.”

/ filters the active pane by substring, which composes naturally with glob tagging: filter to a subset, tag everything visible, then act.

4.5 COMMAND LINE AND HISTORY

The command line is always visible at the bottom of the screen and is focused with **:**. It accepts any registered command name or alias. Several keys are shortcuts that pre-fill it rather than acting directly—**+** opens it containing **tag.glob**, **g** containing **cd**, **F9** containing **mkdir**—so that the relationship between a key and its command is visible at the moment of use rather than only in the help overlay.

History persists across sessions in the cache directory.

4.6 HELP OVERLAY

? opens the help overlay of Figure 2. It is generated from the registry and the key map at runtime. Because both are data, the overlay reflects user key rebindings and cannot become inconsistent with the code.

5. NAVIGATION AND FILE OPERATIONS

5.1 NAVIGATION

Table 7 lists the navigation keys. **Enter** is context-sensitive: on a directory it descends, on an image it opens a viewer, and in the run catalog it plots the run under the cursor.

Table 7. Navigation keys

Key	Action
Tab	Switch the active pane
Arrow keys, j k	Move the cursor
Enter	Descend into a directory, open an image, or plot a catalog row
Backspace	Ascend one directory
g	Prompt for a path
G	Full-screen directory-tree browser
/	Filter the active pane by substring
=	Synchronize the inactive pane to the active pane's directory
Ctrl+U	Swap the two panes
Ctrl+O	Maximize or restore the active pane

5.2 FILE OPERATION SEMANTICS

File operations follow the Norton Commander convention: they act on the active pane's selection, and the default destination is the *inactive* pane's working directory. Table 8 lists the function-key assignments.

Table 8. Function-key file operations

Key	Operation	Behavior
F2	Rename	In-place rename of the file under the cursor.
F3	View	Read-only view in the opposite pane; Esc or a second F3 closes it.
F4	Edit	Opens \$EDITOR.
F5	Refresh	Re-reads both directory listings.
F6	Copy	Selection into the inactive pane, after confirmation.
F7	Move	As copy, then removes the source.
F8	Delete	Always confirms; uses <code>send2trash</code> where available.
F9	Make directory	Prompts in the active pane.
c	Catalog toggle	Flips the inactive pane between file list and run catalog. ^a

^a The catalog toggle is bound to **c** rather than **F10** because most terminal emulators intercept **F10** for their own menu bar. The classic copy, move, and delete bindings are correspondingly shifted from **F5–F7** to **F6–F8** to make room for refresh.

Two behaviors deserve specific mention because they were adopted after user feedback. First, after a delete the cursor remains adjacent to the deleted entry—on the row that took its place, or on the new last row if the deleted file was last—rather than jumping to the top of the list. Cleaning up several files individually is otherwise punishing. Second, all destructive operations confirm before acting, and all are logged.

5.3 REFRESH BEHAVIOR

Operations performed inside the application—copy, move, rename, make directory, batch extract, archive creation—refresh the affected panes automatically. **F5** is the manual fallback for changes made by an external agent: a second shell session writing into a pane’s directory, an autoreduction job depositing new output, or Network File System (NFS) attribute caching catching up.

The mask editor is a hybrid case. It runs as a detached subprocess so the interface stays responsive, but its launcher awaits the exit code in a worker; an exit indicating a save triggers a refresh of both panes so the new mask file appears without user action.

5.4 ARCHIVING AND EMAIL

z creates a zip archive of the selection, prompting for the archive name and defaulting to the inactive pane’s directory. A gzip-compressed tar alternative is available as `archive.tar_gz` on the command line. **e** opens a dialog that sends the selection as attachments by shelling out to `mail` or `mutt`, whichever is configured.

5.5 SHELL ESCAPE

`shell.run`, aliased to `!`, runs an arbitrary shell command with the interface suspended, restoring it when the command exits. This is the escape hatch for the operations the program does not implement.

6. PLOTTING

6.1 PLOTTING THE SELECTION

Plotting is invoked with a single key, **p**, on the active selection: the tagged files if there are any, otherwise the file under the cursor. The program decides what to draw from the file itself rather than asking, so the same keystroke serves every kind of SANS data. Table 9 summarizes what each kind produces.

Table 9. What p draws for each kind of file

Selection	Result
Iq.dat, 2–4 columns	Logarithmic $I(q)$ plot, with error bars when a σ_I column is present. Several files overlay on shared axes.
trans.txt	Transmission $T(\lambda)$ on linear axes, λ in Å.
Iqxqy.dat, 4 or 6 columns	Two-dimensional heat map; several files tile into one window.
*.nxs.h5 raw event file	Detector heat map of summed counts.
*.nxs processed Mantid file	Detector heat map through the same view.
Any other tabular file (l)	Linear plot using the header row as axis labels.

Plots open in their own windows, so the terminal stays usable while a figure is on screen and several figures can be open at once. On a host with no display the figure is instead written as a Portable Network Graphics (PNG) under `~/.cache/sansdir/plots/` and the path is reported in the status bar, so the same keystroke works over a plain SSH session with no forwarding.

Classification does not rely on the file extension alone, which matters because Mantid writes processed output as `.nxs` without the `.h5` suffix and reduced files are not always named predictably. A file whose contents are HDF5 is treated as NeXus whatever it is called, and column count and header keywords disambiguate the ASCII kinds.

6.2 ONE-DIMENSIONAL PLOTS

Reduced $I(q)$ files are drawn on logarithmic axes by default, with error bars when a third column is present. Tagging several files overlays them on shared axes with a filename legend, which is the common comparison case: the same sample at two temperatures, or the same measurement at two detector distances. Figure 4 shows a representative single-curve plot.

Transmission files are treated differently: linear axes, wavelength in Å on the abscissa, and transmission on the ordinate, with labels and title to match.

6.3 TWO-DIMENSIONAL PLOTS

Four- and six-column $I(q_x, q_y)$ files are drawn as a heat map on proper q_x and q_y axes. A logarithmic color scale is the default, which for scattering data is almost always what is wanted.

Tagging more than one two-dimensional file tiles them into a single window with the filename as each subplot title. Two color-bar modes are available. In *shared* mode a single scale spans all panels, so intensities are directly comparable; in *independent* mode each panel is scaled to itself, so weak patterns remain visible. Shared is the default, on the reasoning that side-by-side panels are usually being compared. The choice is made through the `[ui].colorbar_mode` configuration key, which the **p** key reads as a standing preference; a single plot can override it from the command line with `:plot.iqxqy colorbar_mode=independent`, whose argument tab-completes to the two valid values.

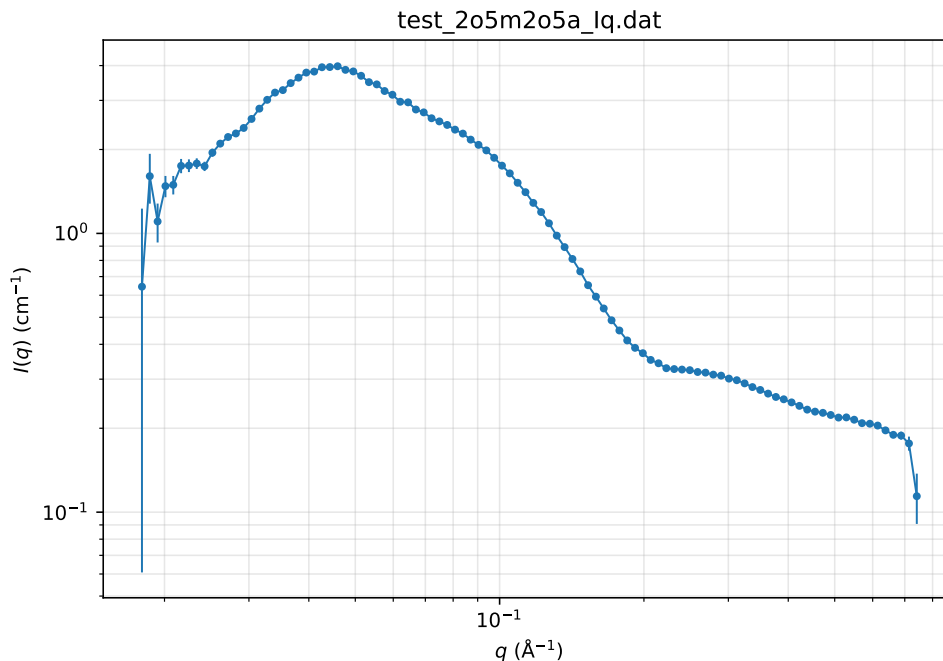


Figure 4. One-dimensional reduced data. A three-column $I(q)$ file plotted on logarithmic axes with error bars. Axis labels and units are applied automatically. This figure was produced by the same code path the **p** key invokes.

6.4 DETECTOR HEAT MAPS FROM NEXUS

Raw EQ-SANS (Zhao, Gao, and Liu 2010) event files are rendered as a 256×192 detector image, as shown in Figure 5. Processed Mantid files—including wavelength-banded `drt sans` output and event-workspace masks—render through the same view.

The image is reconstructed from the event arrays without invoking Mantid, so the detector view stays available on hosts where no Mantid environment is installed or where one is mid-upgrade.

6.5 GENERIC TABULAR PLOTS

l plots any whitespace-, comma-, or tab-delimited table on linear axes, taking axis labels from the header row. This is the companion to the batch metadata extractor of Section 7: extract temperature against time across a series of runs, then plot the resulting table without leaving the program.

Static images—PNG, JPEG, TIFF, and similar—open in a matplotlib window on **Enter**, which is convenient for the figures deposited by autoreduction.

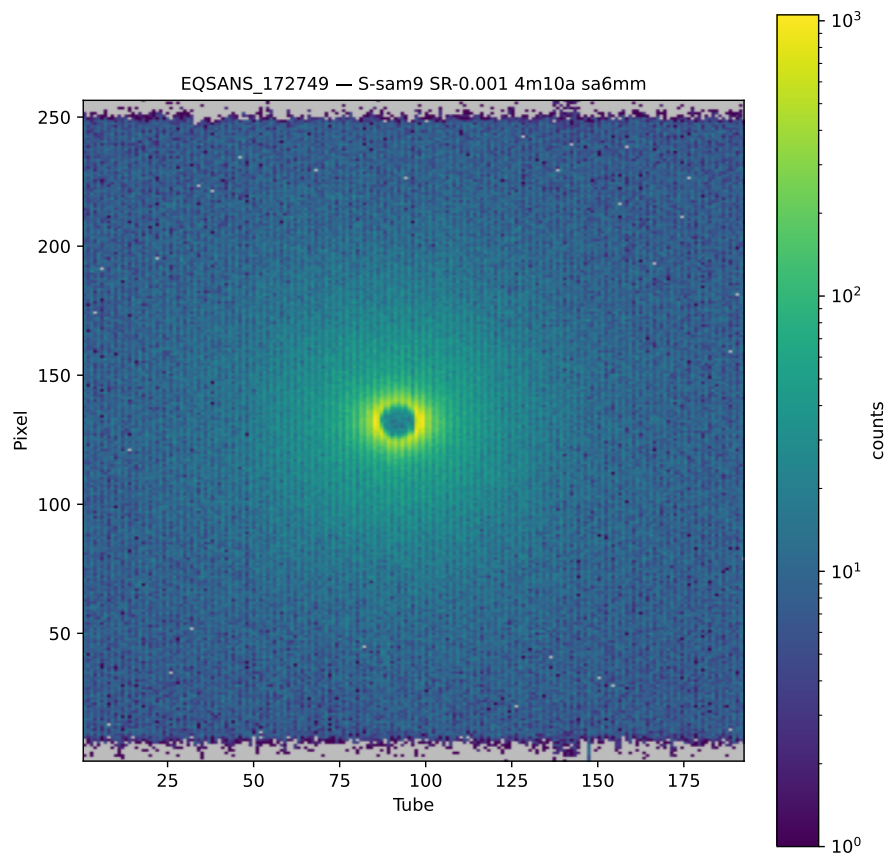


Figure 5. Detector heat map from a raw NeXus event file. Counts summed over all events and reordered into the physical tube-and-pixel layout, on a logarithmic color scale. The beam stop is visible at the center of the direct beam. The reconstruction is performed entirely in `numpy`; no Mantid runtime is involved.

7. METADATA INSPECTION AND EXTRACTION

7.1 SINGLE-FILE TREE BROWSER

m opens the file under the cursor in a modal tree browser. The hierarchy expands lazily: only the direct children of an expanded group are read. This is necessary rather than merely efficient—an exhaustive walk of a several-hundred-megabyte raw file yields tens of thousands of entries and takes seconds, which is both slow and unreadable.

Selecting a leaf displays its full path, dtype, shape, units taken from the `units` attribute, and a one-line value preview.

7.2 KEYWORD SEARCH

Lazy expansion solves the loading problem but creates a navigation one. Sample environment logs live at paths such as `/entry/DASlogs/BL6:SE:QNTC1:ReadProbeTemperature/value`, and reaching one by expanding groups requires knowing the instrument’s device naming scheme.

Pressing `/` in the tree browser therefore switches to keyword search. The full key list is walked once, on a worker thread, and cached; subsequent queries filter it by case-insensitive substring. Results are shown as a table of matching paths with their shapes, and the detail pane follows the highlighted row, so **Up** and **Down** scan candidates without leaving the search box. Figure 6 shows the result of searching a real EQ-SANS file for `temperature`: 28 matches across three distinct devices, located without any knowledge of the naming convention.

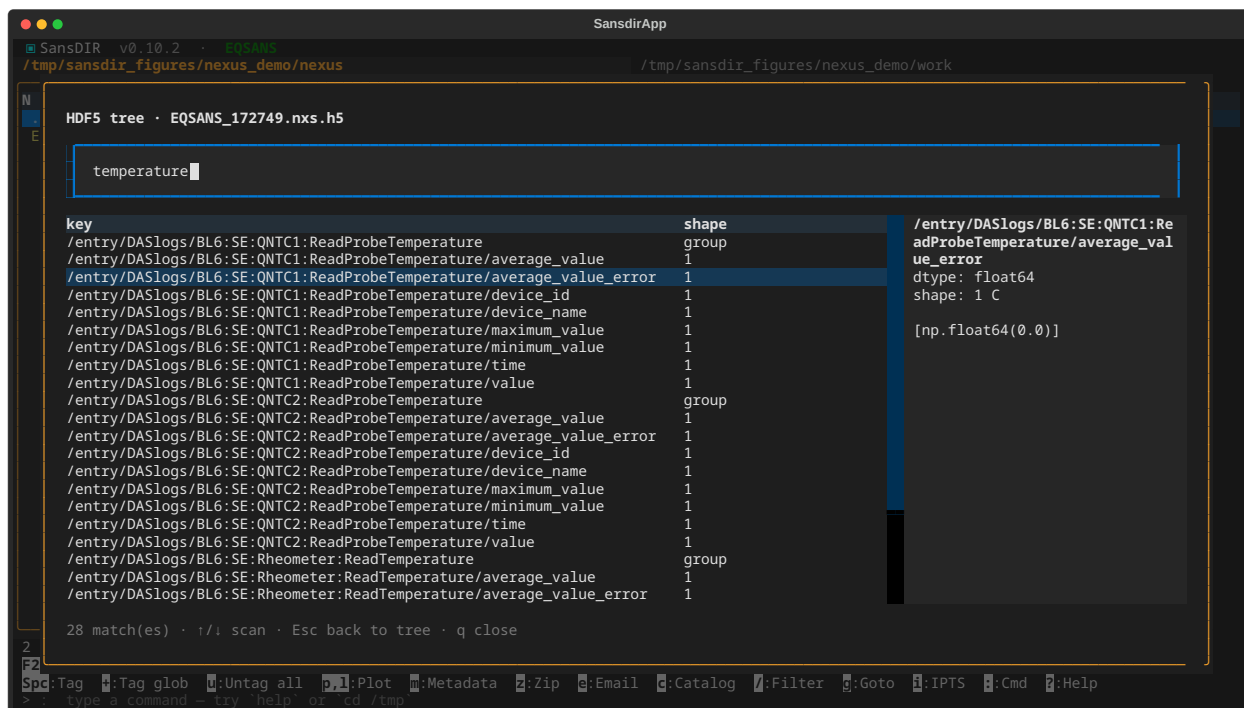


Figure 6. Keyword search in the HDF5 tree browser. A substring query against a raw EQ-SANS file.

The left table lists matching keys with their shapes; the right pane shows the highlighted key’s dtype, shape, and value preview. **Esc** returns to the tree view, and a second **Esc** closes the browser.

Two bounds keep the feature responsive on large files. Rendered matches are capped at 500, with the hint line stating explicitly when a query was truncated rather than silently discarding the remainder. Value previews slice before materializing, so previewing a multi-million-element event array reads five values rather than the whole array.

7.3 BATCH EXTRACTION

M extracts selected keys from many files at once. The dialog opens with a full-screen key picker in which **Space** toggles a key and **/** searches across all keys in the file, using the same search mechanism as the single-file browser. **Ctrl+S** returns to the output form.

Extraction is parallel: **h5py** releases the interpreter lock during reads, so a thread pool—eight workers by default—scales nearly linearly with file count.

The key picker accepts a group as well as a dataset when the group contains a **value** child, which is the SNS DAS log convention. Selecting the group is equivalent to selecting `<group>/value`, which is what the user means.

7.4 OUTPUT MODES

Two output modes cover the two distinct questions users ask.

Summary mode produces one row per input file, with time-series values reduced to their mean. This answers “what was the temperature for each of these 40 runs.” Optional `_stdev` and `_n` columns report the spread and sample count behind each mean.

Per-file mode produces one table per input file with the full arrays preserved. This answers “how did temperature evolve during this run.” It is selected by including the placeholder `<filename>` in the output template, for example `<filename>_temp.csv`. If the user requests per-file mode but omits the placeholder, the program prepends `<filename>_` rather than overwriting one file repeatedly.

Output is written to the inactive pane’s working directory by default, because the active pane is typically the read-only proposal directory and defaulting there would produce a permission error on every attempt.

7.5 A WORKED EXAMPLE

Figure 7 walks through a complete extract-and-plot session on a real experiment file, exactly as a user at the terminal would see it. The file is a Mantid-processed workspace from an EQ-SANS measurement with an in-situ mechanical load frame, and the quantity of interest is the load-cell record from that sample environment—a DAS log named `BL6:SE:PsyloTech:SGLC`—which the user wants as a plottable time series.

1. Navigate the active pane to the file (`IPTS-37828/shared/output/.../*_processed.nxs`) and point the other pane at the directory that should receive the output, here the proposal’s writable `shared/` folder.
2. Press **M**. The key picker opens on the file’s HDF5 tree. Nobody remembers a full DAS log path, so press **/** and type a fragment of the name: `LC`. Three rows match; **Space** on `.../SGLC/time` and `.../SGLC/value` stars both, and the status line confirms *selected: 2 key(s)* (panel a).
3. Press **Ctrl+S**. The picker folds back into the output form with both keys filled in. Per-file mode and the `<filename>_extracted` template are already the defaults; switch the format to comma-separated values (CSV) and press **Ctrl+S** again to run (panel b).

- The extractor writes <filename>_extracted.csv—two columns, time and value, 5 644 rows—into the inactive pane's directory, and that pane refreshes with the cursor ready to move onto the new file (panel c).
- Press **Tab** to switch panes, put the cursor on the new CSV, and press **1**. The generic linear plotter reads the header row for axis labels and draws value against time (panel d): the load ramps during the first 450 seconds of the run and releases sharply—precisely the kind of sanity check that would otherwise require exporting the file and opening a separate analysis tool.

The four panels were captured from the running program, so fonts, colors, and layout are what the terminal actually shows.

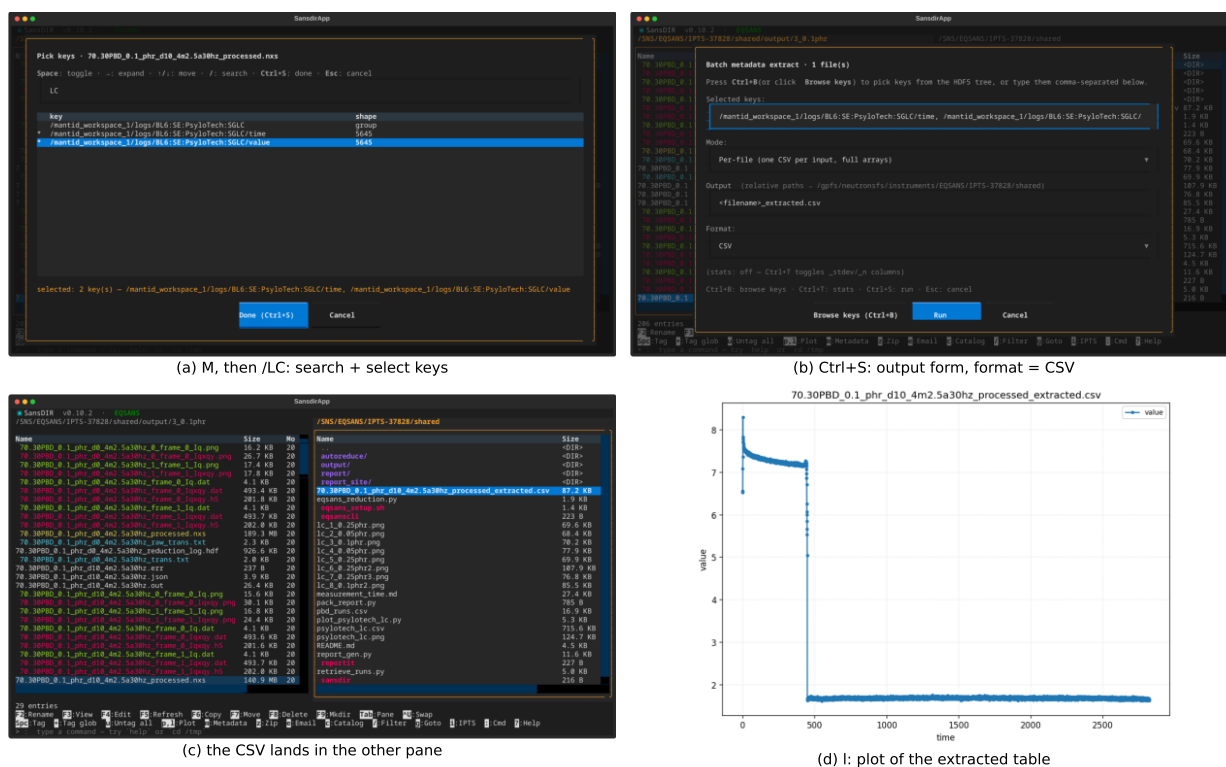


Figure 7. Batch metadata extraction, end to end. (a) The key picker after pressing **M** and searching for LC: both datasets under the PsychoTech: SGLC log are selected. (b) The output form with the chosen keys, per-file mode, and CSV format. (c) The destination pane after the run: the extracted CSV is under the cursor. (d) The curve produced by pressing **1** on it—the load-cell trace over the course of the run.

8. CATALOG INTEGRATION

8.1 AUTHENTICATION

OnCat requires an Open Authorization version 2 (OAuth2) bearer token even for read-only endpoints. The client obtains one through the client-credentials grant and caches it until shortly before expiry. Credentials come from the [oncat] configuration section or from the ONCAT_CLIENT_ID and ONCAT_CLIENT_SECRET environment variables, with working defaults so that a cluster user gets a functioning search without configuration. The endpoint is likewise configurable, so the client can be pointed at a staging instance.

8.2 SEARCH MODEL AND CACHING

OnCat exposes no fuzzy keyword endpoint. The client therefore lists all experiments for the configured instrument once, caches the listing on disk for a configurable time to live (TTL) defaulting to 24 hours, and filters client-side on substring matches against the proposal identifier, the title, and the member list. The first search pays one server round trip; every subsequent search is served from cache and completes in well under a second.

Because a stale cache can hide a newly allocated proposal, **r** in the browser forces a cache-bypassing refetch.

The browser keeps typing responsive on large catalogs in two ways. The filter input is debounced with a 200 ms quiet window, so a fast typist causes one rebuild rather than one per keystroke. The rendered result set is capped at 200 rows, with an overflow hint reporting how many further matches exist.

8.3 RUN CATALOG PANE

Selecting a proposal changes the active pane into that proposal's `shared/` directory and loads its run list into a catalog view on the right pane. **c** toggles the catalog. Within it, **Space** tags runs and the familiar keys operate on the selection: **Enter** or **p** plots the raw detector image, **m** opens the HDF5 tree, **M** extracts metadata, and **K** opens the mask editor. The catalog is thus a third selection surface sharing the same command vocabulary as the two file panes.

9. DETECTOR MASK CREATION

9.1 ARCHITECTURAL CONSTRAINTS

Detector masking is the one feature that produces an input to reduction rather than a view of its output. Three constraints bound the implementation:

- **No Mantid runtime imports in the package.** Searching the source tree for Mantid imports returns nothing. Mask *creation* is pure Python. A Mantid environment is invoked only as an external subprocess, and only at save time, as described below.
- **No instrument-specific geometry in the source.** There is no per-instrument pixel-to-detector formula. The detector mapping is recovered from the source NeXus file itself, from its event-identifier-to-detector correspondence.
- **Pixel ordering is defined in exactly one place.** The invariant that the flattened mask array aligns element-for-element with the pixel identifier array is established in a single module, and the writers consume that invariant rather than re-deriving it.

The masking convention follows Mantid: 1 denotes a masked, excluded pixel and 0 a kept one, so that passing the output to `MaskDetectors` excludes exactly the cells the user drew. Inverting the final mask is a single flag.

9.2 INTERACTIVE EDITOR

K, from a file pane or the run catalog, opens a matplotlib window showing the detector heat map of the selected file. The user draws shapes on it.

The cell aspect ratio is set to 1/1.3 to compensate for the EQ-SANS detector’s tube pitch of roughly 5.2 mm against a pixel pitch of roughly 3.9 mm. Without this correction an ellipse drawn to look circular on screen is not circular on the detector. The canvas extends a few cells beyond the detector boundary, which is marked with a faint dotted line, so that a rubber-band selection can be dragged from outside the active area—otherwise masking the outermost row requires landing the first click exactly on pixel zero.

Table 10 lists the editor’s controls.

Table 10. Interactive mask editor controls

Key	Action
r	Rectangle draw mode
e	Ellipse draw mode
v	Edit mode: click to select a shape, drag to move it, Delete to remove it
k	Open the bank and tube specification dialog
z	Undo
i	Invert the mask
s	Save, through a file chooser
Esc	Quit without saving

The bank and tube dialog accepts a compact specification language—**b3** for a whole bank, **t50** for a tube, **b5-7 t10-15** for mixed ranges—which covers the strip-masking case far better than freehand drawing. The resulting rectangles are ordinary shapes and round-trip through the mask log like any other.

Two responsiveness measures are worth recording because both replaced an implementation that was correct but unusable. Moving a shape in edit mode snapshots the static canvas on mouse-down and re-blits only the moving patch on each motion event, rather than redrawing the full heat map. And the bank and tube input is a dialog rather than an inline matplotlib text widget, because that widget redraws on every keystroke, which on a logarithmically scaled 256×192 image is visible per-character lag.

A cursor readout reports `tube`, `pixel`, `counts`, `bank`, and the tube index within its bank as the pointer moves, which is what allows a user to translate a visual feature into a bank and tube specification.

9.3 COMMAND-LINE MASK CREATION

The same functionality is available non-interactively. Shapes are given in pixel coordinates and the options are repeatable:

```
sansdir mask /SNS/EQSANS/IPTS-XXXXX/nexus/EQSANS_172749.nxs.h5 \  
--circle 96,128,12 \  
--rect 0,0,15,15 --rect 176,0,191,15 \  
--rect 0,240,15,255 --rect 176,240,191,255 \  
--output beam_stop.nxs
```

Available shapes are `-rect X0,Y0,X1,Y1`, `-ellipse XC,YC,RX,R`, `-circle XC,YC,R`, and `-polygon X1,Y1,X2,Y2,...` with at least three vertices. A sidecar log in JavaScript Object Notation (JSON) form is written alongside every output, and `-shapes-json` replays it, so a mask developed interactively can be reapplied to other runs in a script.

9.4 OUTPUT FORMATS AND REDUCTION COMPATIBILITY

Three output formats are supported: a Mantid SaveMask Extensible Markup Language (XML) file, a NeXus file, and a `numpy` array with a metadata sidecar.

The NeXus path deserves explanation because it changed during development. To be usable by the `drtans` reduction pipeline, the mask must carry per-detector mask flags in the instrument parameter map that the pipeline reads. Producing that structure reliably means letting Mantid write it. The save step therefore shells out to the cluster's `drtans` wrapper to run a load-mask-save sequence against the source file. When Mantid is unavailable, the program falls back to a pure-`numpy` writer that produces a visualization-only file, and says so: the fallback output can be inspected but does not affect reduction.

The on-disk encoding of the visualization file mirrors the convention of existing EQ-SANS beam-stop files, in which each *unmasked* detector carries one synthetic event and masked detectors carry none. Consequently a mask opened with `p` renders with the masked region in grey, the same visual convention as a beam stop. The builder retains Mantid's "1 means masked" convention internally; only the on-disk encoding is inverted.

10. NON-INTERACTIVE COMMAND-LINE INTERFACE

Three operations are available without the TUI, for use in scripts and batch jobs. Because they dispatch through the same registry, their behavior matches the interactive equivalents exactly.

10.1 LAUNCHING THE INTERFACE

```
sansdir                                # TUI in the current directory
sansdir /SNS/EQSANS/IPTS-12345/shared  # TUI rooted at a directory
```

A bare path argument is interpreted as `sansdir tui PATH`, so the subcommand name may be omitted in the common case.

10.2 BATCH METADATA EXTRACTION

```
# Summary table: one row per file, time series reduced to means.
sansdir extract \
  -k /entry/DASlogs/temperature/value \
  -k /entry/duration \
  --out summary.tsv \
  /SNS/EQSANS/IPTS-12345/nexus/EQSANS_*.nxs.h5

# Per-file tables: each input gets its own CSV with full arrays.
sansdir extract \
  -k /entry/DASlogs/temperature/time \
  -k /entry/DASlogs/temperature/value \
  --out '<filename>_temp.csv' \
  EQSANS_172749.nxs.h5 EQSANS_172750.nxs.h5

# Add spread and sample-count columns (summary mode only).
sansdir extract --with-stats -k /entry/DASlogs/temperature/value *.nxs.h5
```

Options are `-k/-keys` (repeatable, required), `-o/-out`, `-format` with values `tsv`, `csv`, or `columns`, `-with-stats`, and `-workers` to size the thread pool. The written paths are echoed to standard output, one per line, so the command composes with shell pipelines.

10.3 MASK CREATION

The mask subcommand is documented in Section 9. Its options are `-rect`, `-ellipse`, `-circle`, `-polygon`, `-shapes-json`, `-inverse`, `-output`, and `-format` with values `nxs`, `xml`, or `npz`. On completion it reports the output path, the masked pixel count as an absolute number and a percentage, and the sidecar log path.

11. CONFIGURATION

Configuration is read from `~/.config/sansdir/config.toml`, or from the path named by `$SANSDIR_CONFIG`. Every section and key is optional; the program runs correctly with no configuration file at all.

```
[ui]
theme = "monokai"      # any Textual theme name
colorbar_mode = "shared"  # tiled 2-D scale: "shared" or "independent\"

[keys]
"ctrl+y" = "view.toggle_hidden"  # rebind any key to any command
f5 = "ui.move_tagged"            # unknown commands are dropped at startup

[oncat]
default_instrument = "EQSANS"
cache_ttl_seconds = 86400

[instrument]
default      = "EQSANS"  # "" falls back to [oncat].default_instrument
auto_detect = true      # a /SNS/USANS/... launch path overrides 'default'

[usans]
pixi_manifest      = "/usr/local/pixi/usansred"
reduce_command     = ""  # explicit override for the engine argv
data_dir_template = "/SNS/USANS/{ipts}/shared/autoreduce"
logbin             = true
thickness_cm       = 0.1
short_name_copy    = true  # also write UN*_bsub.txt aliases

[mail]
command = "mail"          # or "mutt"
default_subject = "[sansdir] data"
```

The loader is deliberately tolerant. An unknown theme name falls back to the default with a notification rather than aborting startup, and a key rebinding naming a nonexistent command is dropped with a notification. A malformed configuration file should never prevent the program from launching, because a user who cannot launch the program also cannot use it to fix the file.

Themes may also be changed at runtime with `:theme <name>`; issuing `:theme` with no argument lists the available names.

12. USANS REDUCTION

USANS overlaps almost entirely with what the program already does: the same proposal catalog, the same file manager, the same one-dimensional plotting, the same editor. What it adds is a short reduction workflow and, downstream of it, a desmearing step. The reduction itself is delegated to the instrument team’s `usansred` package; the desmearing implements the truncated Abel inversion of Huang (Huang et al. 2026). Rather than ship a second application, the program carries an *instrument mode* and a small, self-contained module.

12.1 INSTRUMENT MODE

The mode is either SANS or USANS, and it is resolved once at launch: path detection first, then `[instrument].default`, then `[oncat].default_instrument`. Any launch path containing `usans`—which covers `/SNS/USANS/IPTS-N` as well as a user’s own scratch directories—selects USANS. Path detection outranks the configuration file because a user who opens a USANS proposal directory means it, whatever their defaults say. The command `instrument.set`, reached as `:instrument usans`, switches at runtime.

The mode governs four things: which instrument the proposal browser queries, which columns the run catalog shows (USANS has no moveable detector and a fixed incident wavelength, so the distance and wavelength columns would be constants and are dropped), what **Enter** does on a catalog row, and whether the `r` binding is live. It is signalled in the title bar and by the presence of `r:Reduce` in the key hint bar.

Mode scoping is applied to *bindings*, not to registrations. The USANS commands are registered in both modes, so `:usans-reduce` works whether or not the user remembered to switch; only the key disappears. Correspondingly, `default_keymap()` called with no argument returns exactly the map it returned before USANS existed, which is what the help overlay, the hint bar, and the existing test suite all observe.

12.2 THE SETUP TABLE

The reduction engine is driven by a small CSV listing, one row per sample: a flag distinguishing the empty-cell background from the samples, a name that becomes the output filename stem, a first run number, a scan count, a thickness, and optionally runs to exclude. The program generates this table from the proposal’s run list and then gets out of the way: the user reviews and corrects it with **F4**, the same editor used for any other file. There is deliberately no table-editing widget.

Generation groups consecutive runs sharing a title into blocks and picks the block whose title names an empty cell or banjo as the background. The load-bearing step is the scan count. A titled block of N consecutive runs is $(N - 1)$ analyzer rocking-curve scans plus one transmission run, and the engine reads only the rocking-curve files. N is not a constant: it varies with the experiment’s settings. Passing the run count that the catalog reports therefore makes the engine fail on the transmission run’s absent input. The program instead derives the count from which runs actually have rocking-curve files on disk, which handles any block size without configuration, and expresses any gap inside a block through the CSV’s exclude column.

Some judgements cannot be made from the data alone. A measurement that was stopped and restarted under one title leaves an oversized block in which only the last few runs are real; the generator narrows to the last window of the modal size and flags the block. Every such judgement—narrowed blocks, unusual sizes, blocks with no reducible runs, the transmission runs that were dropped, the background that was chosen—is written to a companion notes file. The generated table is a first draft, and the notes exist so that reviewing it is quick rather than archaeological.

12.3 INVOKING THE ENGINE

The reduction mathematics belongs to the instrument team's `usansred` package, deployed on the cluster as a `pixi` environment. The program locates its console script and runs it as a subprocess; it does not reimplement the physics, and no code from `usansred` is copied into this code base. Two deployment realities are absorbed on the caller's side.

First, the console script inherits the invoking user's `~/ .local` site packages and aborts on a stale `pandas`, which is reachable on the analysis nodes. Every invocation therefore disables user site packages and empties `PYTHONPATH`.

Second, the engine resolves its per-run input relative to the directory holding the setup CSV, not the working directory. The reviewed table normally lives in the user's own folder, and the pre-processed data lives under the proposal's autoreduce directory—which is instrument data the program must not write to, and which is frequently read-only to the user in any case. Rather than copy the table into it, the program builds a temporary directory of symbolic links to the data, places a copy of the table there, and runs the engine in that directory. Nothing beneath the instrument data root is modified.

12.4 ONE KEY

The two operations are exposed through a single binding. **r** reduces the setup table under the cursor; when the cursor is not on one, it offers to build a table for the proposal named by the current path, fetching the run list, showing the catalog beside the user, and leaving the cursor on the file it wrote.

It stops there rather than reducing immediately. The table is a first draft whose guesses the notes exist to expose, so the second **r** is the review step, and it costs one keystroke. The one case that does *not* offer to regenerate is a table that parses but fails validation—two backgrounds, a zero scan count: there the user is mid-edit on a real table, and regenerating would discard the corrections. That case reports the problem and points at **F4**.

Reduced output is ordinary two- or three-column text, so the existing plotting path handles it with only a filename rule added to the classifier. The rule matches on the name rather than the column count so that a sample whose name happens to contain `trans` is not routed to the transmission plotter.

The engine's final curve carries a verbose suffix, `_background_subtracted.txt`. Because that name is long, the program writes a shorter `_bsub.txt` alias beside it after each reduce, keeping the original so the engine's own summary spreadsheet and any downstream tooling that expects the standard name continue to work. The alias is a copy, not a symbolic link, and is controlled by `[usans].short_name_copy`.

12.5 A WORKED EXAMPLE

Figure 8 shows the complete flow on a real proposal (IPTS-37679), from an empty working folder to reduced curves, using only the **r** key and the prompts it raises.

1. Open the program in a writable working folder anywhere under the proposal's path. The `/SNS/USANS` prefix puts it in USANS mode automatically.
2. Press **r**. The folder holds no setup table yet, so the program offers to build one for the IPTS named by the path (panel a). Confirm with **y**.
3. The run list is fetched from the catalog and displayed in the other pane, and a prompt asks for the first run of the experiment—here 49434 (panel b). Blocks recorded before that run—beam-cycle alignment and test scans—are noted but left out of the table.

4. The program writes IPTS-37679_setup.csv and its companion NOTE.md, leaves the cursor on the table, and previews its content in the other pane (panel c): here one background and 23 sample blocks. This is the review gate—check the background pick and block sizes against the NOTE, editing with **F4** if needed.
5. Press **r** again, now on the table. The data directory defaults to the proposal's autoreduce folder and the output directory to output/ beside the table; accepting both runs the engine over every block—seconds, since it works from the pre-processed per-run ASCII. On completion the program offers to show the output folder in the other pane (panel d), where each sample now has its UN_*_det_1_* curve family. **p** plots any of them, and **d** desmeares (Section 12.6).

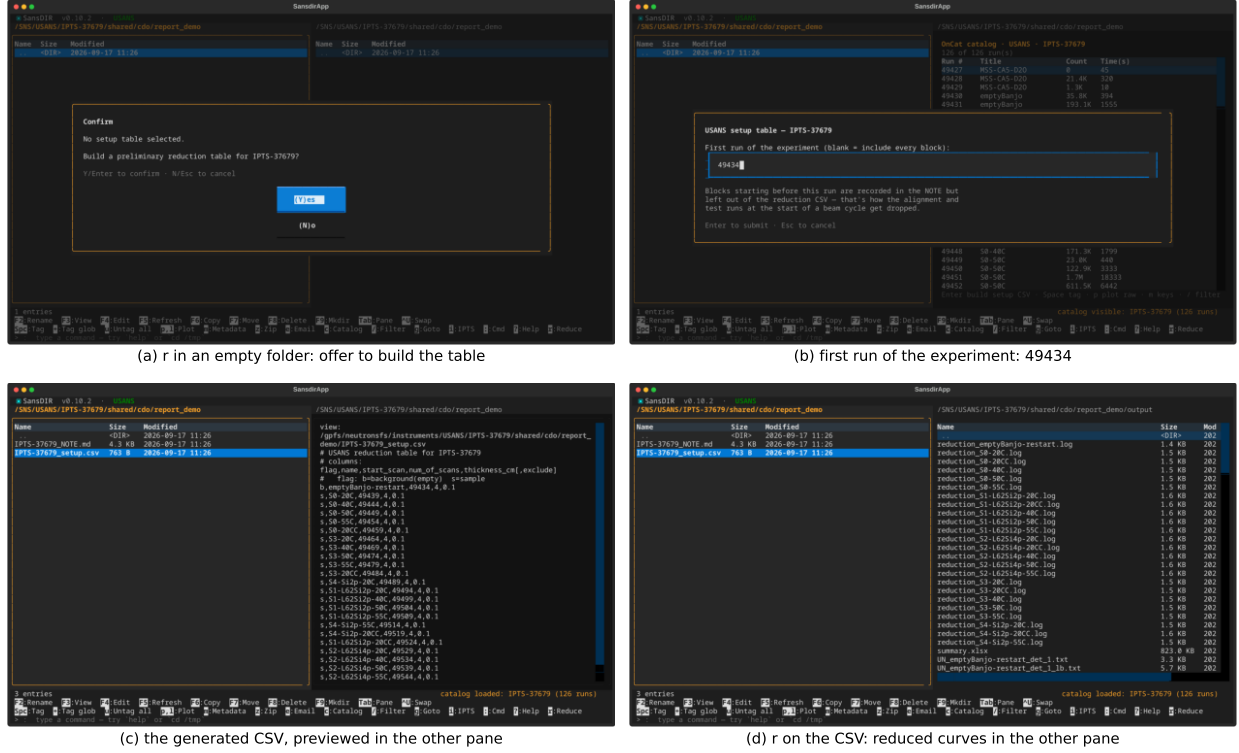


Figure 8. USANS reduction, end to end, on one key. (a) **r** in an empty working folder offers to build the setup table for the IPTS named by the path. (b) The start-of-experiment prompt (run 49434); the fetched run list is visible in the other pane. (c) The generated table under the cursor, previewed in the other pane for review. (d) After the second **r**: the reduced curves in the output folder.

12.6 DESMEARING

USANS instruments use slit rather than pinhole collimation: the larger illuminated area buys the flux that the very high angular resolution of the Bonse–Hart crystal optics demands. The cost is that the recorded intensity at a nominal momentum transfer Q is an average of the true intensity over a tall vertical acceptance,

$$I_{\text{exp}}(Q) = \frac{1}{\sigma_y} \int_0^{\sigma_y} I(\sqrt{Q^2 + s^2}) ds, \quad (1)$$

where σ_y is the vertical resolution half-width. For the SNS instrument $\sigma_y \approx 0.13 \text{ \AA}^{-1}$, two decades above the top of the measured window. Recovering $I(Q)$ from $I_{\text{exp}}(Q)$ is the desmearing problem, and it is ill-posed: the slit average couples each measured point to scattering at much higher Q .

The **d** key desmeares the selected reduced curves. The implementation is the non-iterative truncated Abel inversion of Huang (Huang et al. 2026), which expresses the intrinsic intensity in closed form as

$$I(Q) = I(Q_m) - \frac{2\sigma_y}{\pi} \int_Q^{Q_m} \frac{I'_{\text{exp}}(Q_x)}{\sqrt{Q_x^2 - Q^2}} dQ_x, \quad Q_m = \sqrt{Q^2 + \sigma_y^2}. \quad (2)$$

Being closed-form, it propagates uncertainty through a single linear operator rather than accumulating it over iterations, which is the principal weakness of the widely used Lake algorithm (Lake 1967) it replaces. Every integral is evaluated with the substitution $Q_x = \sqrt{Q^2 + s^2}$, which removes the $1/\sqrt{Q_x^2 - Q^2}$ singularity at the lower limit. A Bayesian Gaussian-process alternative to the same inversion has been developed by Tung (Tung et al. 2026); the program implements the Abel method for its determinism and its closed-form error propagation.

Equation (2) evaluates I at $Q_m \simeq \sigma_y$ and integrates out to there, so it requires the scattering far beyond the USANS window. This drives the one interactive decision in the workflow. When a single curve is selected, the program asks—in the pane the user is already in, while the other pane stays a live, filterable file list—whether a companion pinhole SANS measurement of the same sample is available. If one is chosen, the two are put on a common smearing basis, scale-matched, joined across the instrument gap, and inverted over the full range. If the user declines, or when several curves are desmeared at once, the high- Q side is instead extrapolated as a power law out to σ_y —the approach taken by the National Institute of Standards and Technology (NIST) IGOR/IRENA package (Kline 2006)—and the result is truncated to the measured USANS range, with the extrapolation recorded in the output file’s header. Huang (Huang et al. 2026) show that without genuine high- Q data the reconstruction is reliable only at low Q ; the truncation and the header note make that limitation explicit rather than silent.

The desmeared curve is written as an ordinary three-column Q, I, dI text file that the **p** key plots like any other. The inversion is pure `numpy`, with no new dependency, and was validated by a smear–desmear round trip against analytic power laws: over the core of the USANS window the recovered log–log slope is within half a percent and the median intensity error is below one percent, while the reported uncertainty widens toward the top of the range where the high- Q extrapolation loses authority.

13. SUPPORTED DATA FORMATS

13.1 ASCII REDUCED DATA

Reduced data files are whitespace-delimited ASCII with optional comment lines introduced by #. Table 11 gives the recognized column layouts. Detection counts the columns on the first non-comment line.

Table 11. Recognized ASCII column layouts

Kind	Columns	Layout	Treatment
$I(q)$	2	$q, I(q)$	Logarithmic axes
	3	$q, I(q), \sigma_I$	Logarithmic axes with error bars
	4	$q, I(q), \sigma_I, \sigma_q$	As three-column; the resolution column is not plotted
Transmission	2	$\lambda, T(\lambda)$	Linear axes, λ in Å
	3	$\lambda, T(\lambda), \sigma_T$	With error bars
$I(q_x, q_y)$	4	q_x, q_y, I, σ_I	Heat map on the recovered q grid
	6	$q_x, q_y, I, \sigma_I, dq_x, dq_y$	As four-column

Note: Classification does not rely on column count alone; the filename and any header keywords are considered first.

13.2 NEXUS FILES

Raw event-mode files follow the SNS NeXus conventions (NeXus International Advisory Committee 2025). The paths the program relies on are `/entry/instrument/bank<N>/` for detector data, `/entry/DASlogs/<name>/value` for sample environment values, and `/entry/DASlogs/<name>/time` for the corresponding time axis. Scalars such as `/entry/duration` are read directly.

Processed Mantid files are also accepted for the detector view and for metadata extraction. Since Mantid writes these with a `.nxs` extension and no `.h5` suffix, classification checks the HDF5 magic signature rather than trusting the name.

14. TESTING AND QUALITY ASSURANCE

14.1 TEST SUITE

The suite comprises approximately 800 tests across 40 test modules, of which all pass and two are skipped as environment-dependent (they require a real NeXus file or the installed reduction engine). It runs in about 140 s. Five tests are interface snapshot comparisons.

The suite is layered to match the architecture:

- **Unit tests** cover every module under `core/`, `hdf/`, `mask/`, and the plot classification logic.
- **Registry tests** verify duplicate-name rejection, parameter validation, argument checking at dispatch, and—importantly—that every key binding names a command that exists.
- **Interface tests** drive the real application through Textual’s pilot harness, pressing keys and asserting on resulting state. The feature of Section 7.2, for example, is covered by tests that press `/`, type a query, wait for the worker, and assert on the resulting match list, the case-insensitivity of the match, the two-stage **Esc** behavior, and the empty-result path.
- **Snapshot tests** compare rendered output against stored references to catch unintended layout regressions.

Two testing constraints are enforced. OnCat tests mock representational state transfer (REST) responses and never contact the real service. No test touches a real cluster data path; fixtures are generated synthetically or drawn from a small set of committed files.

14.2 STATIC ANALYSIS AND STYLE

Linting and formatting use `ruff` at a line length of 100, with the E, F, W, I, B, UP, N, C4, SIM, and RUF rule sets enabled. Type checking uses `mypy` in strict mode over `src/sansdir`. Public functions carry type hints and Google-style docstrings. Bare `except` clauses are prohibited: exceptions are caught specifically, logged, and surfaced to the user in the status bar.

CI runs the suite on Python 3.10, 3.11, and 3.12 on Linux.

15. PERFORMANCE

Table 12 states the design budgets. Startup is measured; the remaining figures are the targets against which regressions are judged, and profiling is performed with `pyinstrument` when one is missed.

Table 12. Performance budgets

Operation	Budget	Status
Cold start to first paint	300 ms	~120 ms median ^a
Render a directory of 1,000 entries	100 ms	Met
OnCat search, first call	2 s	Met
OnCat search, cached	100 ms	Met
Single metadata key read	50 ms	Met
Batch extract, 100 files, 5 keys	10 s	Met

^a Median end-to-end wall-clock time for `sansdir -help` on an analysis node, re-measured after the USANS reduction and desmearing features were added; the best case is near 60 ms. The figure did not grow with those features because the fast path imports none of the scientific stack (Section 3.6).

Three specific measures address the cases where a naive implementation would miss a budget. Directory listings are cached and keyed by modification time. The OnCat browser debounces its filter input and caps rendered rows. HDF5 value previews slice before materializing, which bounds the cost of previewing an event array to a constant rather than the array length.

16. SUMMARY

sansdir consolidates the file-handling, inspection, and packaging work that surrounds SANS data reduction into a single keyboard-driven terminal application. It requires no display server, starts in about 54 ms, and runs from a shared installation on the ORNL analysis cluster with no per-user setup.

Its distinguishing technical choice is that every user-facing action is a typed command in one registry, and that key bindings, the interactive command line, and the non-interactive CLI are all thin callers of it. This yields a uniform interaction surface, self-documenting help, and testable business logic decoupled from the terminal framework.

Two limitations bound the present version. The detector heat map and the mask editor assume the EQ-SANS detector layout and have been exercised only against EQ-SANS data; the navigation and file-format features are instrument-agnostic but remain unverified on the High Flux Isotope Reactor (HFIR) SANS instruments. And a mask that `drtsans` will honor still requires a Mantid environment at save time, as described in Section 9.

The USANS workflow of Section 12 has been exercised end to end against a single proposal's data. Its grouping and scan-count reconciliation are driven by conventions—consecutive same-title blocks, a trailing transmission run, empty-cell titles naming a banjo—that hold for the experiments examined but are not enforced by the data format. This is why the generated table is presented as a draft for review rather than as an input to be reduced unattended.

Work in progress extends the program in two directions: minting digital object identifiers (DOIs) for tagged file sets through the Zenodo API, and a natural-language layer that would translate a free-text instruction into registry commands for the user to confirm before execution. Both are additions to the existing command surface rather than changes to it.

ACKNOWLEDGMENTS

This research used resources at the SNS, a US Department of Energy (DOE) Office of Science User Facility operated by ORNL. The mask editor's interaction design was revised substantially in response to feedback from the first instrument scientist to use it on production data.

17. REFERENCES

- Arnold, O., J. C. Bilheux, J. M. Borreguero, et al. 2014. “Mantid—Data Analysis and Visualization Package for Neutron Scattering and μ SR Experiments.” *Nuclear Instruments and Methods in Physics Research Section A* 764:156–166. <https://doi.org/10.1016/j.nima.2014.07.029>.
- Collette, Andrew. 2025. *HDF5 for Python (h5py)*. Software. <https://www.h5py.org/>.
- Do, Changwoo. 2025. *eqsanscli: Command-Line Tools for EQ-SANS*. Software. <https://github.com/cw-do/eqsanscli>.
- Harris, Charles R., K. Jarrod Millman, Stéfan J. van der Walt, et al. 2020. “Array Programming with NumPy.” *Nature* 585 (7825): 357–362. <https://doi.org/10.1038/s41586-020-2649-2>.
- Huang, Guan-Rong, Chi-Huan Tung, Pengwen Chen, Gernot Rother, Yingrui Shang, Carrie Y. Gao, Michelle Everett, et al. 2026. “Analytical desmearing of Bonse–Hart ultra-small-angle neutron scattering data via truncated Abel inversion.” *Journal of Applied Crystallography* 59:1083–1093. <https://doi.org/10.1107/S1600576726004693>.
- Hunter, John D. 2007. “Matplotlib: A 2D Graphics Environment.” *Computing in Science & Engineering* 9 (3): 90–95. <https://doi.org/10.1109/MCSE.2007.55>.
- Kline, Steven R. 2006. “Reduction and analysis of SANS and USANS data using IGOR Pro.” *Journal of Applied Crystallography* 39 (6): 895–900. <https://doi.org/10.1107/S0021889806035059>.
- Lake, J. A. 1967. “An iterative method of slit-correcting small angle X-ray data.” *Acta Crystallographica* 23 (2): 191–194. <https://doi.org/10.1107/S0365110X67002440>.
- McGugan, Will. 2025. *Textual: a Rapid Application Development Framework for Python*. Software. <https://textual.textualize.io/>.
- NeXus International Advisory Committee. 2025. *The NeXus Data Format*. Standard. <https://www.nexusformat.org/>.
- Oak Ridge National Laboratory. 2025. *drtans: Data Reduction Toolkit for Small-Angle Neutron Scattering*. Software. <https://github.com/neutrons/drtans>.
- Preston-Werner, Tom, et al. 2021. *TOML: Tom’s Obvious Minimal Language*. <https://toml.io/>. Specification v1.0.0.
- Ronacher, Armin, and The Pallets Team. 2025. *Click: Composable Command Line Interface Toolkit*. Software. <https://click.palletsprojects.com/>.
- The Midnight Commander Development Team. 2025. *GNU Midnight Commander*. Software. <https://midnight-commander.org/>.
- Tung, Chi-Huan, Guan-Rong Huang, Yingrui Shang, Gernot Rother, Changwoo Do, Yangyang Wang, Jan-Michael Carrillo, et al. 2026. “Desmearing Bonse–Hart USANS data using Bayesian Gaussian process regression.” *The Journal of Chemical Physics* 164 (20): 204102. <https://doi.org/10.1063/5.0320684>.
- Zhao, J. K., C. Y. Gao, and D. Liu. 2010. “The Extended Q-Range Small-Angle Neutron Scattering Diffractometer at the SNS.” *Journal of Applied Crystallography* 43 (5): 1068–1077. <https://doi.org/10.1107/S002188981002217X>.

APPENDIX A. COMMAND REGISTRY REFERENCE

APPENDIX A. COMMAND REGISTRY REFERENCE

Table A-1 lists every command registered in version 0.10.2. Any command may be invoked from the `:` line by name or alias.

Table A-1. Registered commands. A trailing question mark marks an optional parameter

Command (alias)	Parameters	Description
<code>app.browse_tree</code>	<code>root?</code>	Open a directory tree browser; the selected directory becomes the active pane's.
<code>app.cmdline_open</code>	—	Focus the command line.
<code>app.cmdline_prompt</code>	<code>text</code>	Open the command line pre-filled.
<code>app.help (help)</code>	—	Show the help overlay.
<code>app.quit (quit, q)</code>	—	Exit.
<code>archive.tar_gz (tar)</code>	<code>srcs, out_path</code>	Create a gzip-compressed tar archive.
<code>archive.zip</code>	<code>srcs, out_path</code>	Create a zip archive.
<code>edit.file (edit)</code>	<code>path</code>	Edit in \$EDITOR.
<code>file.copy</code>	<code>srcs, dst_dir</code>	Copy into a destination directory.
<code>file.delete</code>	<code>paths, trash?</code>	Delete or trash.
<code>file.mkdir (mkdir)</code>	<code>name</code>	Create a directory.
<code>file.move</code>	<code>srcs, dst_dir</code>	Move, or rename a single file.
<code>hdf.batch_extract (extract)</code>	<code>paths, keys, out?, fmt?, with_stats?</code>	Extract log values from many NeXus files into a table.
<code>hdf.show_keys (hdf)</code>	<code>path</code>	Open the Hierarchical Data Format version 5 (HDF5) tree browser.
<code>instrument.set (instrument)</code>	<code>name?</code>	Switch the session between small-angle neutron scattering (SANS) and ultra-small-angle neutron scattering (USANS).
<code>mail.send</code>	<code>recipient, subject?, attachments?, body?</code>	Send mail via the configured client.
<code>nav.cd (cd)</code>	<code>path</code>	Change the active pane's directory.
<code>nav.up</code>	—	Ascend one directory.
<code>oncat.search (ipts)</code>	<code>keyword?, instrument?</code>	Browse OnCat experiments.
<code>pane.activate</code>	<code>panel_id</code>	Activate the named pane.
<code>pane.swap</code>	—	Swap the two panes.
<code>pane.sync</code>	—	Match the inactive pane to the active one.
<code>pane.toggle_catalog</code>	—	Flip the inactive pane between file list and run catalog.
<code>pane.toggle_max</code>	—	Maximize or restore the active pane.
<code>plot.detector_sum</code>	<code>paths</code>	Render NeXus detector banks as heat maps.
<code>plot.generic (plot.linear, lplot)</code>	<code>paths</code>	Linear plot of any tabular file.
<code>plot.image (image, view.image)</code>	<code>paths</code>	Open image files in matplotlib.
<code>plot.iq</code>	<code>paths</code>	Overlay 2-, 3-, or 4-column $I(q)$ files.
<code>plot.iqxqy</code>	<code>paths, colorbar_mode?</code>	Plot 4- or 6-column $I(q_x, q_y)$ files; <code>colorbar_mode</code> is shared or independent.
<code>plot.transmission</code>	<code>paths</code>	Plot $T(\lambda)$ on linear axes.
<code>shell.run (!)</code>	<code>cmd</code>	Run a shell command with the interface suspended.
<code>tag.clear</code>	—	Clear all tags in the active pane.
<code>tag.glob</code>	<code>pattern</code>	Tag visible entries matching a glob.

Table A-1, continued. A trailing question mark marks an optional parameter.

Command (alias)	Parameters	Description
tag.toggle	advance?	Toggle the tag on the cursor row.
tag.untag_glob	pattern	Untag entries matching a glob.
ui.activate_cursor	—	Context-sensitive Enter .
ui.batch_extract	paths?	Open the extraction dialog.
ui.copy_tagged	—	Copy the selection to the inactive pane.
ui.delete_tagged	—	Delete the selection, with confirmation.
ui.mail_tagged	—	Mail the selection as attachments.
ui.mask (mask)	path?	Open the mask editor.
ui.move_tagged	—	Move the selection to the inactive pane.
ui.plot_auto	—	Plot the selection, routing by file kind.
usans.init_table (usans-init)	start_run?, out_dir?, data_dir?, ipts?	Build a preliminary USANS setup comma-separated values (CSV) and notes.
usans.reduce (usans-reduce)	path?, output_dir?, data_dir?, logbin?	Reduce a setup CSV, or offer to build one.
usans.desmear (desmear)	paths?, sans?, out_dir?, sigma_y?	Desmear reduced USANS curves by truncated Abel inversion.
view.in_other_pane	path	Show a file in the other pane (open, not toggle).
ui.plot_generic	—	Linear plot of the selection.
ui.refresh (refresh)	—	Re-read both listings.
ui.rename (rename)	—	Rename the file under the cursor.
ui.set_theme (theme)	name?	Switch theme; no argument lists available names.
ui.zip_tagged	—	Prompt for an archive path and zip the selection.
view.file (view)	path	Open a read-only pager.
view.set_filter (filter)	pattern?	Filter the active pane; empty clears.
view.set_sort	key, reverse?	Set the sort key.
view.toggle_hidden	—	Show or hide dotfiles.
view.toggle_other_pane	—	Show the cursor's file in the other pane.

APPENDIX B. DEFAULT KEY MAP

APPENDIX B. DEFAULT KEY MAP

Table B-1 lists the bindings shown in the help overlay. Ten further bindings exist as alternate terminal encodings of the same keys and are hidden from help.

Table B-1. Default key bindings

Key	Action	Command
Tab	Switch the active pane	pane.activate
Ctrl+U	Swap the two panes	pane.swap
=	Sync the inactive pane	pane.sync
Ctrl+O	Maximize or restore	pane.toggle_max
Enter	Open a directory, image, or text file	ui.activate_cursor
Backspace	Ascend one directory	nav.up
Space	Tag or untag the current row	tag.toggle
+	Tag by glob	app.cmdline_prompt
-	Untag by glob	app.cmdline_prompt
u	Untag all	tag.clear
h	Toggle hidden files	view.toggle_hidden
1 2 3 4	Sort by name, time, size, extension	view.set_sort
F2	Rename	ui.rename
F3	View in the other pane	view.toggle_other_pane
F4	Edit	edit.file
F5	Refresh both panes	ui.refresh
F6	Copy to the other pane	ui.copy_tagged
F7	Move to the other pane	ui.move_tagged
F8	Delete	ui.delete_tagged
F9	Make directory	app.cmdline_prompt
g	Jump to a path	app.cmdline_prompt
G	Browse the filesystem tree	app.browse_tree
/	Filter the active pane	app.cmdline_prompt
c	Toggle the run catalog	pane.toggle_catalog
i	OnCat proposal search	oncat.search
p	Plot the selection	ui.plot_auto
l	Linear plot of the selection	ui.plot_generic
m	HDF5 metadata tree	hdf.show_keys
M	Batch metadata extract	ui.batch_extract
K	Create a detector mask	ui.mask
z	Zip the selection	ui.zip_tagged
e	Email the selection	ui.mail_tagged
:	Open the command line	app.cmdline_open
?	Help overlay	app.help
r	Reduce a USANS setup table (USANS mode only)	usans.reduce
d	Desmear the selected USANS curves (USANS mode only)	usans.desmear
q	Quit	app.quit

APPENDIX C. WORKED EXAMPLE SESSION

APPENDIX C. WORKED EXAMPLE SESSION

The following sequence is representative of a triage session and exercises most of the feature set. It assumes a proposal has been allocated and reduced data exists.

1. Launch with `sansdir`.
2. Press **i** to open the OnCat browser. Press **/** to focus the filter, type a proposal number or a keyword from the title, and press **Enter** to move into the result list. Press **Enter** on the desired proposal and confirm. The active pane moves into the proposal's `shared/` directory and the run catalog loads on the right.
3. Move into the catalog with **Tab**, scroll to a run of interest, and press **p** to view its raw detector image. Confirm the beam center and beam-stop placement, then close the plot window.
4. Press **m** on the same run and then **/** to search for `temperature`. Confirm the sample environment value recorded for the run. Press **Esc** twice to return.
5. Press **Tab** back to the file pane and descend into the reduction output directory.
6. Tag the reduced curves to compare: press **+**, enter a glob such as `*20C*Iq.dat`, and press **Enter**. Press **p** to overlay them on logarithmic axes. Press **u** to clear the tags afterwards.
7. Tag the corresponding transmission files and press **p** again; they are recognized as transmission data and drawn on linear axes.
8. Press **M** to extract metadata across the run series. In the key picker, press **/**, search for the log of interest, press **Space** to select it, and press **Ctrl+S**. Choose summary mode and write the table.
9. Press **l** on the resulting table to plot it directly.
10. Tag the files to share, press **z**, and enter an archive name. Then tag the archive, press **e**, and complete the mail dialog.
11. Press **q** to exit.

